

程式碼實驗室：已簽署的容器映像檔

來源：<https://codelabs.developers.google.com/signed-container-image-codelab?hl=zh-tw>

步驟數：5

在本程式碼研究室中，您將瞭解如何運用已簽署的容器映像檔功能，提高機密空間的可用性。

目錄

1. [1. 總覽](#)
2. [2. 設定雲端資源](#)
3. [3. 建立及簽署工作負載](#)
4. [4. 授權及執行工作負載](#)
5. [5. 清除](#)

1. 總覽

本程式碼研究室以「[Confidential Space 程式碼研究室](#)」為基礎。支援已簽署的容器映像檔，並提供選項，可使用經過驗證的公開金鑰驗證容器，而不必在 Workload Identity Pool (WIP) 政策中指定映像檔摘要。

Confidential Space 的簽署容器映像檔支援功能有何異動：

提升可用性：導入已簽署的容器映像檔功能後，協作者/稽核人員授權映像檔時，現在可以從工作負載映像檔摘要方法改用容器簽章方法。

- 直接使用映像檔摘要時，資源擁有者每次授權新映像檔時，都必須使用映像檔摘要更新政策。使用映像檔簽章時，政策會包含公開金鑰指紋，對應的私密金鑰則由協作者/稽核人員擁有，並用於簽署稽核的映像檔。
- 對於某些安全模式，參照可信的映像檔簽署金鑰，比更新新映像檔摘要值清單更方便。

注意：使用圖片簽章「不會」取代所有使用者的圖片摘要。不過，如果環境中的圖片變更速度很快，或是需要協調許多資源擁有者，可能就比較適合使用這項功能。

不會造成安全性倒退：這個容器簽章方法不會造成安全性倒退，因為信任邊界維持不變。在容器簽章方法中，資源擁有者會在 WIP 政策中指定信任的公開金鑰指紋，授權驗證金鑰，授權檢查則由 Attestation Verifier Service 和 WIP 執行；Attestation Verifier Service 會驗證簽章是否與執行中的工作負載相關聯，WIP 政策則會檢查服務聲明的公開金鑰是否已獲得政策授權。

安全性高：使用容器映像檔簽章時，可將部分信任授權給映像檔簽署者。在驗證政策中指定可信簽署者的公開金鑰指紋，資源擁有者即可授權該簽署者，對符合政策的容器映像檔進行背書。驗證服務會驗證簽名是否與執行中的工作負載相關聯，而政策會檢查建立簽名的公開金鑰是否已獲得政策授權。因此，圖片簽署提供的額外間接層可維持 Confidential Space 的強大安全保障。

這兩種方法的唯一差異在於，後者使用額外的間接層，工作負載映像檔會透過簽署金鑰授權。由於信任邊界維持不變，因此不會產生任何新的安全漏洞。

課程內容

在本程式碼研究室中，您將瞭解如何使用容器映像檔簽章授權存取受保護的資源：

- 如何使用 `cosign` 簽署經過稽核的容器映像檔
- 如何將容器映像檔簽章上傳至 OCI 登錄檔，以供探索及儲存簽章
- 如何設定執行 Confidential Space 時所需的雲端資源
- 如何透過簽署的容器映像檔支援功能，在 Confidential Space 中執行工作負載

本程式碼實驗室說明如何使用 Confidential Space，對在 Google Compute Engine 上執行的容器映像檔進行遠端認證，該映像檔已使用受信任的金鑰簽署。

軟硬體需求

- 完成 [Confidential Space 程式碼研究室](#)
- Google Cloud Platform 專案
- Chrome 或 Firefox 等瀏覽器
- 熟悉標準 Linux 文字編輯器，例如 Vim、Emacs 或 Nano
- 具備 [Sigstore cosign](#) 的基本知識

- Google Compute Engine ([程式碼研究室](#))、[機密 VM](#)、[容器](#)和遠端存放區的基本知識
- Cloud KMS 基礎知識 ([程式碼研究室](#))
- [服務帳戶](#)、[Workload Identity Federation](#)和[屬性條件](#)的基本知識。
- Artifact Registry 的基本知識
- 數位簽章的基本知識

使用已簽署容器映像檔的機密空間所涉及的角色

在本程式碼研究室中，Primus Bank 將擔任稽核人員和資源擁有者，負責下列事項：

1. 使用樣本資料設定必要資源。
2. 稽核工作負載程式碼。
3. 使用 `cosign` 簽署工作負載映像檔。
4. 將簽章上傳至存放區。
5. 設定 WIP 政策，保護客戶資料。

Secundus Bank 將是工作負載的作者和運算子，負責：

1. 設定儲存結果所需的資源。
2. 編寫工作負載程式碼。
3. 發布工作負載映像檔。
4. 在 Confidential Space 中執行工作負載，並支援已簽署的容器映像檔。

Secundus Bank 將開發及發布工作負載，查詢 Primus Bank 擁有的雲端儲存空間 bucket 中儲存的顧客資料。Primus Bank 會稽核工作負載、簽署容器映像檔，並設定 WIP 政策，允許核准的工作負載存取資料。這項工作負載執行的結果會儲存在 Secundus 銀行擁有的 Cloud Storage bucket 中。

設定 Confidential Space 時涉及的資源

本程式碼研究室會參照多個變數，您應為 GCP 專案設定適當的值。本程式碼研究室中的指令會假設這些變數已設定。(例如，`export PRIMUS_INPUT_STORAGE_BUCKET='my-input-bucket'` 可用於設定 Primus 銀行輸入儲存空間 bucket 的名稱)。如果資源名稱的變數尚未設定，系統會根據 GCP 專案 ID 產生變數。

在 Primus 專案中設定下列項目：

- `$PRIMUS_INPUT_STORAGE_BUCKET`：儲存客戶資料檔案的 bucket。
- `$PRIMUS_WORKLOAD_IDENTITY_POOL`：驗證聲明的 Workload Identity Pool (WIP)。
- `$PRIMUS_WIP_PROVIDER`：Workload Identity Pool 提供者，其中包含授權條件，可用於 Attestation Verifier Service 簽署的權杖。
- `$PRIMUS_SERVICEACCOUNT`：`$PRIMUS_WORKLOAD_IDENTITY_POOL` 用於存取受保護資源的服務帳戶。在這個步驟中，該服務帳戶有權查看儲存在 `$PRIMUS_INPUT_STORAGE_BUCKET` 值區中的客戶資料。
- `$PRIMUS_ENC_KEY`：用來加密 `$PRIMUS_INPUT_STORAGE_BUCKET` 中所儲存資料的 KMS 金鑰。

本程式碼研究室的新資源：

- `$PRIMUS_COSIGN_REPOSITORY`：用於儲存工作負載映像檔簽章的 Artifact Registry。
- `$PRIMUS_SIGNING_KEY`：稽核人員/資料協作者 (在本例中為 Primus Bank) 用於簽署工作負載映像檔的 KMS 金鑰。

在 Secundus 專案中設定下列項目：

- `$SECUNDUS_ARTIFACT_REGISTRY`：要推送工作負載 Docker 映像檔的 Artifact Registry。

- `$WORKLOAD_IMAGE_NAME`：工作負載 Docker 映像檔的名稱。
- `$WORKLOAD_IMAGE_TAG`：工作負載 Docker 映像檔的標記。
- `$WORKLOAD_SERVICEACCOUNT`：有權存取執行工作負載的機密 VM 的服務帳戶。
- `$SECUNDUS_RESULT_BUCKET`：儲存工作負載結果的 bucket。

其他資源：

- `primus_customer_list.csv` 包含顧客數位資料。我們會將這項資料上傳至 `$PRIMUS_INPUT_STORAGE_BUCKET`，並建立查詢這項資料的工作負載。

現有工作流程

在 Confidential Space 中執行工作負載時，系統會使用設定的資源執行下列程序：

1. 工作負載會從 WIP 要求 `$PRIMUS_SERVICEACCOUNT` 的一般 Google 存取權杖。這項服務會提供工作負載和環境聲明，以及認證驗證器服務權杖。
2. 如果 Attestation Verifier 服務權杖中的工作負載評估聲明與 WIP 中的屬性條件相符，系統會傳回 `$PRIMUS_SERVICEACCOUNT` 的存取權杖。
3. 工作負載會使用與 `$PRIMUS_SERVICEACCOUNT` 相關聯的服務帳戶存取權杖，存取 `$PRIMUS_INPUT_STORAGE_BUCKET` 值區中的客戶資料。
4. 工作負載會對該資料執行作業。
5. 工作負載會使用 `$WORKLOAD_SERVICEACCOUNT` 服務帳戶，將該作業的結果寫入 `$SECUNDUS_RESULT_STORAGE_BUCKET` 值區。

支援已簽署容器的新工作流程

簽署容器支援功能將整合至現有工作流程，如下方醒目顯示的部分所示。在 Confidential Space 中執行工作負載時，如果支援已簽署的容器映像檔，系統會使用設定的資源執行下列程序：

1. **Confidential Space** 會探索與目前執行中工作負載映像檔相關的任何容器簽章，並將這些簽章傳送至驗證驗證器。認證驗證器會驗證簽章，並在認證聲明中加入所有有效簽章。
 2. 工作負載會從 WIP 要求 `$PRIMUS_SERVICEACCOUNT` 的一般 Google 存取權杖。這項服務會提供工作負載和環境聲明，以及認證驗證器服務權杖。
 3. 如果認證驗證器服務權杖中的容器簽章權杖附加資訊，與 WIP 中的屬性條件相符，則會傳回 `$PRIMUS_SERVICEACCOUNT` 的存取權杖。
 4. 工作負載會使用與 `$PRIMUS_SERVICEACCOUNT` 相關聯的服務帳戶存取權杖，存取 `$PRIMUS_INPUT_STORAGE_BUCKET` 值區中的客戶資料。
 5. 工作負載會對該資料執行作業。
 6. 工作負載會使用 `$WORKLOAD_SERVICEACCOUNT`，將該作業的結果寫入 `$SECUNDUS_RESULT_STORAGE_BUCKET` 值區。
-

步驟 2 · 約 0 分鐘

2. 設定雲端資源

在設定 Confidential Space 時，您首先會在 Primus 和 Secundus 銀行 GCP 專案下建立必要的雲端資源。以下是本程式碼研究室的新資源：

在 Primus 專案中：

- 稽核程式碼後，用於簽署 Secundus 工作負載的 KMS 簽署金鑰。
- 用於儲存 Cosign 簽章的 Artifact Registry 存放區。

Secundus 專案中沒有新資源。設定這些資源後，您將為工作負載建立服務帳戶，並指派必要角色和權限。接著，您會建立工作負載映像檔，稽核員 (Primus 銀行) 則會簽署該映像檔。接著，資料協作者 (本程式碼研究室中的 Primus 銀行) 會授權工作負載，工作負載運算子 (本例中的 Secundus 銀行) 則會執行工作負載。

設定 Confidential Space 時，您會在 Primus 和 Secundus GCP 專案中建立必要的雲端資源。

事前準備

- 使用下列指令複製 [這個存放區](#)，取得本程式碼研究室所需的指令碼。

```
git clone https://github.com/GoogleCloudPlatform/confidential-space
```

- 變更本程式碼研究室的目錄。

```
cd confidential-space/codelabs/signed_container_codelab/scripts
```

- 請確認您已設定必要專案，如下所示。

```
export PRIMUS_PROJECT_ID=<GCP project id of primus bank>
export SECUNDUS_PROJECT_ID=<GCP project id of secundus bank>
```

- 使用這項指令，為上述資源名稱設定變數。您可以使用這些變數覆寫資源名稱 (例如 `export PRIMUS_INPUT_STORAGE_BUCKET='my-input-bucket'`)
- 執行下列 [指令碼](#)，根據資源名稱的專案 ID，將其餘變數名稱設為值。

```
source config_env.sh
```

- 請按照 [這裡](#) 的說明安裝 cosign。

設定 Primus 銀行資源

在這個步驟中，您將為 Primus 銀行設定必要的雲端資源。執行下列 [指令碼](#)，為 Primus 銀行設定資源。在這些步驟中，系統會建立下列資源：

- Cloud Storage bucket (`$PRIMUS_INPUT_STORAGE_BUCKET`)，用於儲存 Primus 銀行加密的顧客數位資料檔案。
- KMS 中的加密金鑰 (`$PRIMUS_ENC_KEY`) 和金鑰環 (`$PRIMUS_ENC_KEYRING`)，用於加密 Primus 銀行資料檔案。
- 工作負載身分集區 (`$PRIMUS_WORKLOAD_IDENTITY_POOL`)：根據提供者設定的屬性條件驗證權杖附加資訊。
- 附加至上述 workload identity pool (`$PRIMUS_WORKLOAD_IDENTITY_POOL`) 的服務帳戶 (`$PRIMUS_SERVICEACCOUNT`)，具備下列 IAM 存取權：

- `roles/cloudkms.cryptoKeyDecrypter`，使用 KMS 金鑰解密資料。
- `objectViewer`，從 Cloud Storage 值區讀取資料。
- `roles/iam.workloadIdentityUser`，將這個服務帳戶連結至工作負載身分集區。

```
./setup_primus_bank_resources.sh
```

設定 Secundus 銀行資源

在這個步驟中，您將為 Secundus 銀行設定必要的雲端資源。執行下列[指令碼](#)，為 Secundus 銀行設定資源。按照下列步驟操作時，系統會建立下列資源：

- Cloud Storage bucket (`$SECUNDUS_RESULT_STORAGE_BUCKET`)，用於儲存 Secundus 銀行執行工作負載的結果。

```
./setup_secundus_bank_resources.sh
```

3. 建立及簽署工作負載

建立工作負載服務帳戶

現在，您要為工作負載建立服務帳戶，並指派必要角色和權限。執行下列**指令碼**，在 Secundus 銀行專案中建立工作負載服務帳戶。執行工作負載的 VM 會使用這個服務帳戶。

- 這個工作負載服務帳戶 (`$WORKLOAD_SERVICEACCOUNT`) 將具備下列角色：
- `confidentialcomputing.workloadUser` 取得驗證權杖
- `logging.logWriter`，將記錄檔寫入 Cloud Logging。
- `objectViewer`，從 `$PRIMUS_INPUT_STORAGE_BUCKET` Cloud Storage bucket 讀取資料。
- `objectAdmin`，將工作負載結果寫入 `$SECUNDUS_RESULT_STORAGE_BUCKET` 雲端儲存空間 bucket。

```
./create_workload_serviceaccount.sh
```

建立工作負載

您將在本步驟中建立工作負載 Docker 映像檔。本程式碼研究室使用的工作負載是簡單的 CLI 型 Go 應用程式，可計算引數中指定地理位置的客戶 (來自 Primus 銀行客戶資料)。執行下列**指令碼**，建立工作負載，其中會執行下列步驟：

- 建立 Secundus 銀行擁有的 Artifact Registry(`$SECUNDUS_ARTIFACT_REGISTRY`)。
- 使用必要資源名稱更新工作負載程式碼。[這裡](#)是本程式碼研究室使用的工作負載程式碼。
- 建構 Go 二進位檔，並建立 Dockerfile，用於建構工作負載程式碼的 Docker 映像檔。本程式碼研究室使用的 Dockerfile [如下](#)。
- 建構 Docker 映像檔，並發布至 Secundus 銀行擁有的 Artifact Registry (`$SECUNDUS_ARTIFACT_REGISTRY`)。
- 授予 `$WORKLOAD_SERVICEACCOUNT` 的讀取權限給 `$SECUNDUS_ARTIFACT_REGISTRY`。工作負載容器需要這項權限，才能從 Artifact Registry 提取工作負載 Docker 映像檔。

```
./create_workload.sh
```

簽署工作負載

我們將使用 Cosign 簽署工作負載映像檔。Cosign 預設會將簽章儲存在與簽署映像檔相同的存放區。如要指定其他簽章存放區，可以設定 `COSIGN_REPOSITORY` 環境變數。

我們將以 [Artifact Registry](#) 為例。您也可以根據偏好選擇其他以 [OCI 為基礎](#)的登錄檔，例如 Docker Hub、AWS CodeArtifact。

注意：如果簽署者沒有工作負載存放區的寫入權限，強烈建議執行這個步驟。

1. 建立 Artifact Registry Docker 存放區。

```
gcloud config set project $PRIMUS_PROJECT_ID
```

```
gcloud artifacts repositories create $PRIMUS_COSIGN_REPOSITORY \
  --repository-format=docker --location=${PRIMUS_PROJECT_REPOSITORY_REGION}
```

2. 在 KMS 下建立金鑰環和金鑰，用於簽署工作負載映像檔。

```
gcloud config set project $PRIMUS_PROJECT_ID
```

```
gcloud kms keyrings create $PRIMUS_SIGNING_KEYRING \
  --location=${PRIMUS_PROJECT_LOCATION}
```

```
gcloud kms keys create $PRIMUS_SIGNING_KEY \
  --keyring=$PRIMUS_SIGNING_KEYRING \
  --purpose=asymmetric-signing \
  --default-algorithm=ec-sign-p256-sha256 \
  --location=${PRIMUS_PROJECT_LOCATION}
```

3. 如果是 Artifact Registry，則需要提供完整映像檔名稱，例如 `$LOCATION/$PROJECT/$REPOSITORY/$IMAGE_NAME`。您可以將任何容器映像檔上傳至存放區，以儲存簽章。

```
export COSIGN_REPOSITORY=us-
docker.pkg.dev/${PRIMUS_PROJECT_ID}/${PRIMUS_COSIGN_REPOSITORY}/demo
```

注意：不同登錄檔可能需要不同格式的「存放區」。

- GCR 登錄檔預期格式如下：

```
$ export COSIGN_REPOSITORY=gcr.io/primus-signatures
```

- Artifact Registry 需要完整的映像檔名稱，例如 `$LOCATION-docker.pkg.dev/$PROJECT/$REPOSITORY/$STORAGE_IMAGE`，而不只是存放區。例如：

```
$ export COSIGN_REPOSITORY=us-
docker.pkg.dev/${PRIMUS_PROJECT_ID}/${PRIMUS_COSIGN_REPOSITORY}/demo
```

在 Artifact Registry 中，只指定存放區 (例如 `us-docker.pkg.dev/${PRIMUS_PROJECT_ID}/${PRIMUS_COSIGN_REPOSITORY}`) 無效。

4. 將 `$PRIMUS_COSIGN_REPOSITORY` 存放區的「檢視者」角色授予 `$WORKLOAD_SERVICEACCOUNT` 服務帳戶。這可讓 Confidential Space 探索上傳至 `$PRIMUS_COSIGN_REPOSITORY` 的任何容器映像檔簽章。

```
gcloud artifacts repositories add-iam-policy-binding ${PRIMUS_COSIGN_REPOSITORY} \
  --project=${PRIMUS_PROJECT_ID} --role='roles/viewer' --location=us \
  --
member="serviceAccount:${WORKLOAD_SERVICEACCOUNT}@${SECUNDUS_PROJECT_ID}.iam.gserviceaccount.
com"
```

Cosign 是一項功能強大的工具，提供多種簽署功能。就我們的用途而言，我們只需要 Cosign 使用金鑰組簽署。這項簽署容器映像檔功能不支援 Cosign 無金鑰簽署。

使用金鑰組簽署時，有兩種做法：

1. 使用 Cosign 產生的本機金鑰組簽署。
2. 使用儲存在其他位置 (例如 KMS) 的金鑰組簽署。

注意：支援已簽署容器映像檔的 Confidential Space 僅支援使用 RSA 和 ECDSA 金鑰的 Cosign 簽署。

1. 如果沒有金鑰組，請在 Cosign 中產生。詳情請參閱[使用自行管理的金鑰簽署](#)。我們在此指定了兩種產生金鑰配對和簽署工作負載的方式 (在本機和使用 KMS 提供者)，請按照其中一種方式簽署工作負載容器。

```
// Set Application Default Credentials.
gcloud auth application-default login
```

```
// Generate keys using a KMS provider.
cosign generate-key-pair --kms <provider>://<key>
```

```
// Generate keys using Cosign.
cosign generate-key-pair
```

在上述內容中，將 <provider>://<key> 替換為

```
gcpkms://projects/$PRIMUS_PROJECT_ID/locations/global/keyRings/$PRIMUS_SIGNING_KEYRING/cryptoKeys/$PRIMUS_SIGNING_KEY/cryptoKeyVersions/$PRIMUS_SIGNING_KEYVERSION
```

- <provider>：指您使用的 KMS 解決方案
- <key>：指 KMS 中的金鑰路徑

2. 擷取用於驗證的公開金鑰。

```
// For KMS providers.
cosign public-key --key <some provider>://<some key> > pub.pem

// For local key pair signing.
cosign public-key --key cosign.key > pub.pem
```

3. 使用 Cosign 簽署工作負載。對公開金鑰執行未填補的 Base64 編碼

```
PUB=$(cat pub.pem | openssl base64)
```

```
// Remove spaces and trailing "=" signs.
PUB=$(echo $PUB | tr -d '[:space:]' | sed 's/[=]*$//')
```

注意：我們使用未填補的 [RFC4648 配置](#) 來編碼 PEM 公開金鑰，因為 Cosign 不允許酬載註解中出現任何額外的「=」符號。

4. 使用 Cosign 簽署工作負載，並附加匯出的公開金鑰和簽章演算法。

```
IMAGE_REFERENCE=us-  
docker.pkg.dev/$SECUNDUS_PROJECT_ID/$SECUNDUS_ARTIFACT_REPOSITORY/$WORKLOAD_IMAGE_NAME:$WORKL  
OAD_IMAGE_TAG
```

```
// Sign with KMS support.  
cosign sign --key <some provider>://<some key> $IMAGE_REFERENCE \  
-a dev.cosignproject.cosign/sigalg=ECDSA_P256_SHA256 \  
-a dev.cosignproject.cosign/pub=$PUB
```

```
// Sign with a local key pair.  
cosign sign --key cosign.key $IMAGE_REFERENCE \  
-a dev.cosignproject.cosign/sigalg=ECDSA_P256_SHA256 \  
-a dev.cosignproject.cosign/pub=$PUB
```

- `--key` [必要] 指定要使用的簽署金鑰。如要參照由 KMS 供應商管理的金鑰，請按照「[Sigstore KMS 支援](#)」一文中的特定 URI 格式操作。如要參照 Cosign 產生的金鑰，請改用 `cosign.key`。
- `$IMAGE_REFERENCE` [必要]：指定要簽署的容器映像檔。`IMAGE_REFERENCE` 的格式可透過標記或圖片摘要識別。例如：`us-docker.pkg.dev/$SECUNDUS_PROJECT_ID/secundus-workloads/workload-container:latest` or `us-docker.pkg.dev/$SECUNDUS_PROJECT_ID/secundus-workloads/workload-container [IMAGE-digest]`
- `-a` [必要]：附加至簽章酬載的註解。如果是 Confidential Space 簽署的容器映像檔，簽章酬載必須附上公開金鑰和簽章演算法。

警告：請勿變更 `dev.cosignproject.cosign/pub` 和 `dev.cosignproject.cosign/sigalg`，因為 Confidential Space 只會接受這些金鑰名稱。

- `dev.cosignproject.cosign/sigalg` 僅接受下列三個值：
- `RSASSA_PSS_SHA256`：採用 PSS 填充演算法的 RSASSA 演算法，並使用 SHA256 摘要。
- `RSASSA_PKCS1V15_SHA256`：RSASSA 演算法，搭配 PKCS#1 v1.5 填充演算法和 SHA256 摘要。
- `ECDSA_P256_SHA256`：含有 SHA256 摘要的 P-256 曲線 ECDSA。這也是 Cosign 產生的金鑰組所用的預設簽章演算法。

注意：使用 Cosign 簽署時，請選擇對應的簽章演算法。RSA 金鑰的長度也應至少為 2048 位元。

5. 將簽章上傳至 Docker 存放區

Cosign 會自動將簽名上傳至指定的 `COSIGN_REPOSITORY`。

步驟 4 · 約 0 分鐘

4. 授權及執行工作負載

授權工作負載

在這個步驟中，我們會在 workload identity pool (`$PRIMUS_WORKLOAD_IDENTITY_POOL`) 下方設定 workload identity provider。如下所示，系統會為 workload identity 設定屬性條件。其中一項條件是根據簽署公開金鑰的指紋，驗證工作負載映像檔簽章的指紋。有了這項屬性條件，當 Secundus Bank 發布新的工作負載映像檔時，Primus Bank 會稽核工作負載程式碼並簽署新的工作負載映像檔，不必使用映像檔摘要更新 WIP 政策。

```
gcloud config set project $PRIMUS_PROJECT_ID
```

```
PUBLIC_KEY_FINGERPRINT=$(openssl pkey -pubin -in pub.pem -outform DER | openssl sha256 | cut -d' ' -f2)
```

```
gcloud iam workload-identity-pools providers create-oidc ${PRIMUS_WIP_PROVIDER} \
  --location="global" \
  --workload-identity-pool="${PRIMUS_WORKLOAD_IDENTITY_POOL}" \
  --issuer-uri="https://confidentialcomputing.googleapis.com/" \
  --allowed-audiences="https://sts.googleapis.com" \
  --attribute-mapping="google.subject='assertion.sub'" \
  --attribute-condition="assertion.swname == 'CONFIDENTIAL_SPACE' &&
'STABLE' in assertion.submods.confidential_space.support_attributes
  && '${WORKLOAD_SERVICEACCOUNT}@${SECUNDUS_PROJECT_ID}.iam.gserviceaccount.com' in
  assertion.google_service_accounts
  && ['ECDSA_P256_SHA256:${PUBLIC_KEY_FINGERPRINT}']
  .exists(fingerprint, fingerprint in
assertion.submods.container.image_signatures.map(sig,sig.signature_algorithm+':'+sig.key_id))
"
```

執行工作負載

在這個步驟中，我們會在機密 VM 上執行工作負載。使用中繼資料標記傳遞必要的 TEE 引數。工作負載容器的引數是透過旗標的「`tee-cmd`」部分傳遞。工作負載已編碼，可將結果發布至 `$SECUNDUS_RESULT_STORAGE_BUCKET`。

```
gcloud compute instances create ${WORKLOAD_VM} \
  --confidential-compute-type=SEV \
  --shielded-secure-boot \
  --maintenance-policy=MIGRATE \
  --scopes=cloud-platform \
  --zone=${SECUNDUS_PROJECT_ZONE} \
  --project=${SECUNDUS_PROJECT_ID} \
  --image-project=confidential-space-images \
  --image-family=confidential-space \
  --service-account=${WORKLOAD_SERVICEACCOUNT}@${SECUNDUS_PROJECT_ID}.iam.gserviceaccount.com \
  --metadata "^~^tee-image-reference=us-docker.pkg.dev/${SECUNDUS_PROJECT_ID}/${SECUNDUS_ARTIFACT_REPOSITORY}/${WORKLOAD_IMAGE_NAME}:${WORKLOAD_IMAGE_TAG}~tee-restart-policy=Never~tee-cmd=[\"count-location\\\", \"Seattle\\\", \"gs://${SECUNDUS_RESULT_STORAGE_BUCKET}/seattle-result\"]\"~tee-signed-image-repos=us-docker.pkg.dev/${PRIMUS_PROJECT_ID}/${PRIMUS_COSIGN_REPOSITORY}/demo"
```

查看結果

在 Secundus 專案中，查看工作負載的結果。

```
gcloud config set project $SECUNDUS_PROJECT_ID
```

```
gsutil cat gs://${SECUNDUS_RESULT_STORAGE_BUCKET}/seattle-result
```

結果應為 3，因為 `primus_customer_list.csv` 檔案中列出的西雅圖人數就是這麼多！

5. 清除

這個指令碼可用於清理在本程式碼研究室中建立的資源。在這次清理作業中，系統會刪除下列資源：

- Primus 銀行輸入儲存空間值區 (`$PRIMUS_INPUT_STORAGE_BUCKET`)。
- Primus 銀行服務帳戶 (`$PRIMUS_SERVICEACCOUNT`)。
- Primus Bank 構件登錄檔，其中包含圖片簽章 (`$PRIMUS_COSIGN_REPOSITORY`)。
- Primus Bank workload identity pool (`$PRIMUS_WORKLOAD_IDENTITY_POOL`)。
- Secundus Bank 的工作負載服務帳戶 (`$WORKLOAD_SERVICEACCOUNT`)。
- 工作負載運算執行個體。
- Secundus Bank 的結果儲存空間 bucket (`$SECUNDUS_RESULT_STORAGE_BUCKET`)。
- Secundus Bank 的 Artifact Registry (`$SECUNDUS_ARTIFACT_REGISTRY`)。
- Secundus Bank 的工作負載 VM (`$WORKLOAD_VM`)。

```
./cleanup.sh
```

探索完畢後，請考慮刪除專案。

- 前往 [Cloud Platform Console](#)。
- 選取要關閉的專案，然後按一下頂端的「刪除」，系統就會排定刪除專案的時間。

恭喜

恭喜，您已成功完成本程式碼研究室！

您已瞭解如何運用已簽署的容器映像檔功能，提升 Confidential Space 的可用性。

後續步驟

查看一些類似的程式碼研究室...

- [運用 Confidential Space 保護使用中的共用資料](#)
- [如何透過多方運算和機密空間進行數位資產交易](#)
- [使用 Confidential Space 分析機密資料](#)

其他資訊

- [感到孤單嗎？機密運算來救援](#)
 - [GCP 的機密運算](#)
 - [Confidential Space：保護隱私權的協作方式，引領未來趨勢](#)
 - [Google 和 Intel 如何提升機密運算的安全性](#)
 - [隱私權與進展：運用 Google Cloud 機密運算技術提升安全性](#)
-